

Processi

Arturo Carpi

Dipartimento di Matematica e Informatica
Università di Perugia

Corso di Sistemi Operativi - a.a. 2024/25

Definizione

Un **processo** (o **job**) è un programma in esecuzione

Elementi del processo

- **program counter** e contenuto dei registri (**immagine del processore**)
- **stack** (dati temporanei)
 - variabili locali
 - indirizzi di return
 - parametri delle funzioni
- Programma (**sezione testo**)
- **Sezione dati** (variabili globali)
- **Heap** (allocazione dinamica)

Programma

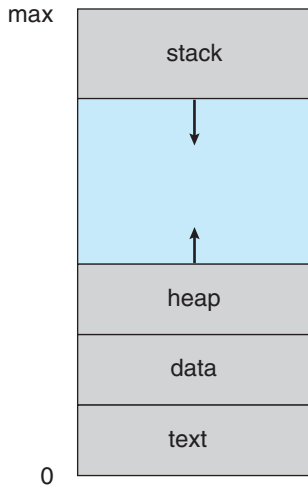
Entità passiva: file eseguibile

Processo

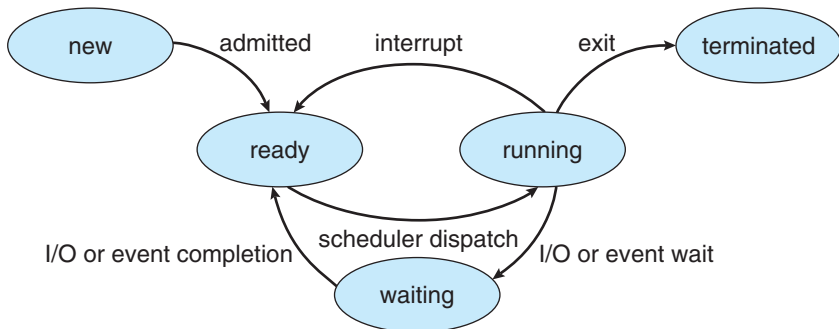
Entità attiva: con PC e risorse associate

Nota: Un programma può essere associato a più processi

Processo in memoria



Stati del processo



Nuovo: In fase di creazione

In esecuzione: Il processore sta eseguendo le istruzioni

In attesa: Il processo attende un evento

Pronto: Il processo attende la disponibilità di un processore

Terminato: Il processo ha completato l'esecuzione

Descrittore di processo (PCB)

A ogni processo è associata una struttura dati detta descrittore di processo (**process control block**), contenente tutte le informazioni relative al processo:

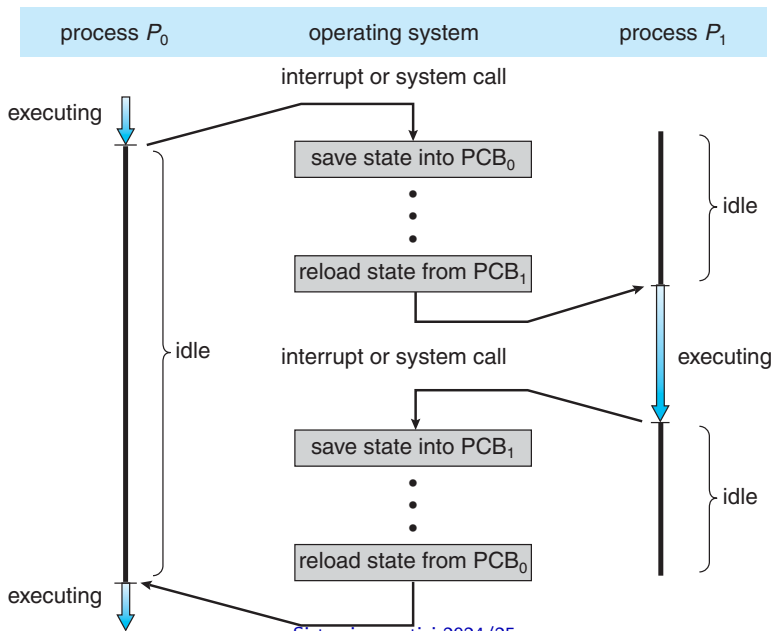
- Stato del processo
- Program counter
- Registri CPU (immagine del processore)
- Parametri per lo scheduling della CPU
- Informazioni per la gestione della memoria
- Informazioni di accounting
- Informazioni su I/O

Threads

Un processo può avere multiple sequenze di istruzioni (**threads**) che possono essere eseguite in parallelo (anche su core differenti) per realizzare **task** differenti.

In tal caso, il PCB include informazioni su ciascun thread.

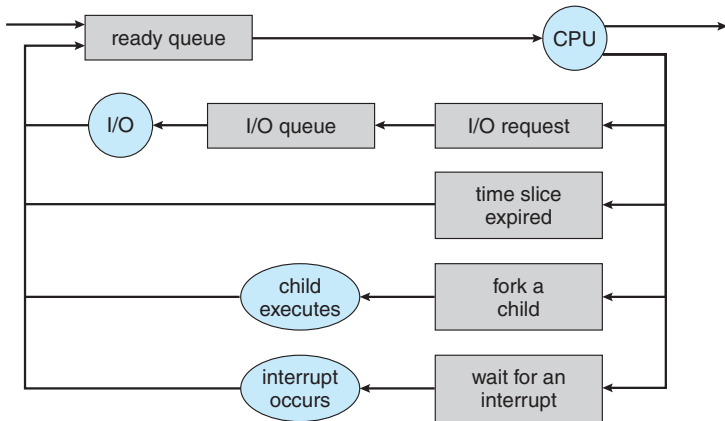
Salvataggio del PCB



Code di schedulazione

I PCB sono inseriti in varie code:

- Coda **job**, contenente tutti i processi,
- Coda **ready**, contenente i processi pronti
- Code dei **dispositivi**, una per dispositivo, contenente i processi in attesa di quel dispositivo



Schedulatore a breve termine

- CPU scheduler
- Assegna la CPU a uno dei processi pronti
- È eseguito con grande frequenza (millisec)
 - Deve essere veloce

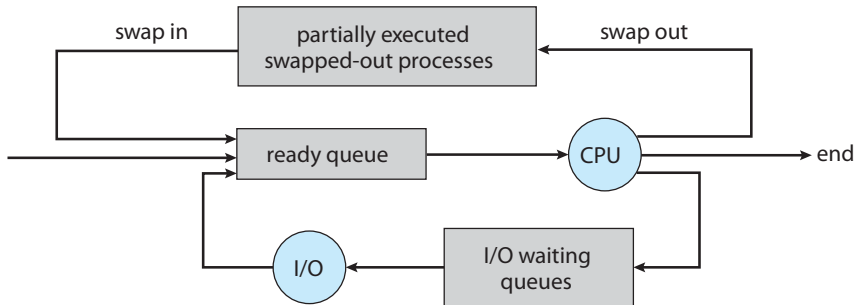
Schedulatore a lungo termine

- job scheduler
- Seleziona i processi da inserire in coda pronti
 - controlla il grado di multiprogrammazione
- Non è eseguito con frequenza
 - Può applicare politiche complesse

Schedulatore a medio termine

- Assegna/revoca la memoria ai processi (**swapping**)

Schedulazione a medio termine



Come selezionare i processi

La schedulazione realizza delle politiche per l'assegnazione della CPU.

Generalmente distingue tra

- Processi I/O-bound
- Processi CPU-bound

Overhead

L'overhead del cambio di contesto varia in funzione di

- velocità della memoria
- numero dei registri
- istruzioni speciali

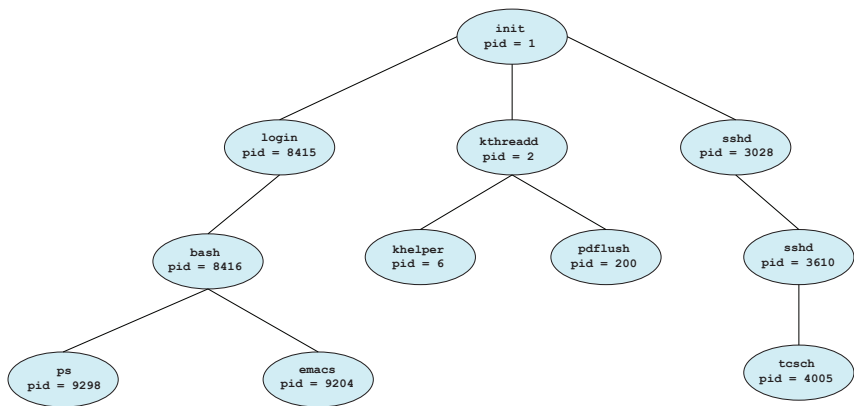
Creazione dei processi

- Il sistema operativo fornisce meccanismi per
 - creazione
 - terminazionedei processi

Creazione del processo

- Un processo **genitore** crea un processo **figlio** che a sua volta può creare altri processi
 - albero dei processi
- Ogni processo è identificato da un numero detto **identificatore di processo** (o **pid**)

Esempio (Linux)



Assegnazione delle risorse

- Il processo figlio ottiene le risorse dal processo genitore
 - Risorse ripartite
 - Risorse condivise
- Il processo figlio ottiene le risorse dal sistema operativo

Opzioni di esecuzione

- Il processo genitore e processo figlio eseguiti concorrentemente
- Il genitore attende la terminazione del figlio

Memoria

- Il figlio è un duplicato del genitore
- Il figlio ha un nuovo programma

```
#include < sys/types.h >
#include < stdio.h >
#include < unistd.h >
int main()
{
    pid_t pid;
    /* fork a child process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }
    return 0;
}
```

- Il processo termina con la chiamata di sistema `exit()`
 - restituisce un valore di stato al processo genitore, via `wait()`
 - le risorse sono deallocate dal sistema operativo
- Il genitore termina l'esecuzione del figlio con la chiamata di sistema `abort()`
 - eccesso di risorse allocate
 - compito non più richiesto
- terminazione a cascata

Deallocazione del pid

Solo dopo il `wait()` del genitore

- zombie
- orfani